Successor and Primitive Recursion from a Scalar Successor-Seeded Iterator

Research prepared for Vladimir Reshetnikov

10 November 2026

Abstract

We resolve a specific open arithmetic problem in the supplied drafts: a successor-seeded iterator whose update receives only its previous state already supports a whole-domain predecessor construction. Its numerical language has only zero and $R(n, a; u.h)$; successor is an abbreviation, and the update has no old-counter argument. Two computation schemas suffice. The key construction first builds a finite permutation exchanging zero and one, uses conjugation to obtain zero-start iteration, and then replaces the usual homogeneous factor map by affine maps adapted to the shifted seed. A strict comparator and saturated iteration give $P(0) = 0$ and $P(Sn) = n$, using only literal-equation induction and numerical cases. Ordinary initialized iteration, full primitive recursion, and an identity-domain bi-interpretation with intuitionistic PRA follow.

The logical kernel is the existing Choice-DI presentation, explicitly specified here; its reduction of intuitionistic connectives is inherited. We separately measure complete checkers against all nine current draft archives at a pinned commit. A smaller numerical basis is not by itself a proof of a smaller complete checker, and no absolute minimality theorem is asserted. All claims and remaining complexity limitations are stated separately from the mathematical equivalence theorem.

Contents

1 The result and the comparison boundary **2**

2 Complete primitive specification **3**

3 A scalar successor-seeded recursor is sufficient **5**

4 From the new iterator to full intuitionistic PRA **10**

5 Proof checking, source cost, and the precise evidence boundary **12**

6 What the construction does and does not minimize **18**

7 Reproduction and contents **19**

A Complementary introductions with one local assumption **19**

B From literal-equation induction to full intuitionistic PRA **25**

C Pairing, primitive recursion, and internal numerical composites **30**

1 The result and the comparison boundary

The new mathematical result is the completion of the one-variable successor-seeded proposal recorded as unresolved in `ipra_shared_continuation_2026-11-10.zip`, Section 9[2]. That draft already states the proposed equations and proves that a left inverse of successor would recover ordinary initialized iteration. It explicitly does not supply such a left inverse. We supply the missing term and an internal proof. Thus the candidate syntax and final initialization compiler are attributed to that draft; the contribution here is the finite-swap and affine-comparator bootstrap between them.

The earlier proved successor-recursion basis[3] has an update $h(u, v)$ receiving both the old counter and the previous state. Its predecessor is immediate from the counter argument. The present update receives only its previous state. Simply identifying those two earlier arguments would invalidate the earlier predecessor proof; the construction below replaces that proof.

Theorem 1.1 (Main mathematical result)**.** *Let $\mathcal{S}$ have the numerical syntax, two computation schemas, and native logical rules in Section 2. There is a finite $\mathcal{S}$ term P such that $\mathcal{S}$ derives*

$$P(0) = 0, \qquad P(Sx) = x.$$

Its ordinary first-order realization is bi-interpretable and equiconsistent with full first-order iPRA*. Both numerical interpretation domains are the entire original domain, equality is actual equality, and the definable isomorphism for each numerical composite is $x = y$.*

Here iPRA uses intuitionistic first-order logic, symbols for fixed primitive-recursive descriptions, their defining equations, the usual successor boundary properties, and quantifier-free induction. The equivalent atomic-induction formulation is standard[7]; the whole-domain equality-reflection argument needed for that equivalence is included, rather than assumed from standard numerical calculations.

1.1 What was actually inspected

The comparison snapshot is

`4a04139595eadc3e8818f1df2a94838ed4851d83`.

All nine top-level ZIPs in `docs/drafts` were retrieved and verified against their Git blob identities and lengths. The recursive scan includes ten embedded archive occurrences: 415 Python member occurrences and 164 distinct source contents, with zero Python parse failures. It inspects historical comparison sources as well as newly advertised files. The inventory and the classification of every source capable of improving any coordinate are included in `evidence/`. Small verification drivers that call an external checker are not themselves complete checker competitors.

Complete source	Tokens	AST nodes	Normalized bytes
Generated assumptions, main	281	318	685
Generated assumptions, AST profile	283	315	710
Guarded elimination, token profile	281	325	739
Current coordinate envelope	281	315	685

The envelope consists of independently attained upper bounds. It is not a mathematical lower bound on all proof checkers. No implementation can be certified absolutely simplest without a fixed implementation language, cost convention, and a separate minimality argument.

Complete new program	Tokens	AST nodes	Normalized bytes
Generated-source scalar R	279	319	686
Byte profile	279	321	681
AST profile	282	314	708
New coordinate envelope	**279**	**314**	**681**

The new family is strictly below all three old coordinates: by 2 tokens, 1 AST node, and 4 normalized bytes. Different complete programs attain the coordinates. *No single delivered program is below all three old minima simultaneously.* These are small measured implementation improvements, separate from the new predecessor theorem. The new arithmetic packet protocol generates the zero/successor source counter; ordinary import and driver choices give the separate profiles. The same packet change also improves an initialized-I control, so the source savings are not attributed to the smaller numerical basis itself. Section 5 gives exact matched comparisons.

2 Complete primitive specification

2.1 Numerical syntax and exactly two computation schemas

There is one nonempty numerical sort. Terms are finite staged descriptions

$$t ::= x \mid 0 \mid R(n, a; u.h).$$

Only the body h binds u; the counter n and initializer parameter a do not. All other free variables of h are passive numerical parameters. The body is fixed finite earlier syntax. Define the abbreviations

$$Sa := R(0, a; u.0), \qquad 1 := S0.$$

The complete primitive numerical computation inventory is

$$R(0, a; u.h) = Sa, \tag{R0}$$
$$R(Sn, a; u.h) = h[R(n, a; u.h)/u]. \tag{RS}$$

Every substitution is capture avoiding, after renaming bound variables. Expanding the abbreviation S, R0 reads $R(0, a; u.h) = R(0, a; u.0)$; the counter on the left of RS is $R(0, n; u.0)$. Consequently no successor symbol is concealed as an additional primitive constructor.

For ordinary first-order elaboration, introduce one function symbol $r_h(n, a, \vec{p})$ for each fixed body $h(u, \vec{p})$. Actual arguments may coincide, but declared formal parameters are distinct. There is no sort of all numerical functions, no numerical quantification over program descriptions, and no uniformly total evaluator. Closure under a finite description is a syntactic schema, not an object-language higher-type existence axiom.

2.2 Native formulas, contexts, and all logical rules

Native formulas are

$$A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A.$$

The bound variables in D are distinct. Contexts are finite reusable lists of formulas. Native absurdity is the designated equation $0 = S0$. The intended ordinary readings are

$$(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ(y), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x (B^\circ(x,0) \to \forall y B^\circ(x,y)).$$

An equation translates to the same actual equality. The following is the entire rule inventory, in addition to R0 and RS as computation leaves:

$$\frac{}{\Gamma \vdash A}\ (A \in \Gamma), \qquad \frac{\Gamma \vdash s = t \quad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}, \qquad \frac{\Gamma \vdash \mathsf{Q}_y B(y) \quad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}. \qquad \text{(Hyp, Tr, QE)}$$

Here F is any fixed native formula template. Equality transport does not authorize induction on its motive. Paired elimination is

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x,y) \quad \Gamma, \mathsf{Q}_y B(a,y) \vdash A}{\Gamma \vdash A}. \qquad \text{(DE)}$$

The parameter a is fresh for Γ, A and all passive parameters of B. Paired introduction has the two choices

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}, \qquad \frac{\Gamma, E(t,z) \vdash E(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} E(x,y)}. \qquad \text{(DI-case, DI-progress)}$$

In DI-progress, E must be a literal equation, not an equation under a binder. DI-case allows every matrix. The parameter z is fresh for Γ, t and the passive matrix parameters. All displayed local assumptions are discharged. There are no other primitive logical axioms, successor axioms, coverage axioms, reflexivity axioms, or induction rules. Weakening, exchange, contraction, renaming, numerical substitution, and ordinary-assumption cut are admissible by induction on derivations.

The direct formula-context use of Q is a conservative presentation of the inherited closure-clause interface. A closure is interderivable with $\mathsf{D}_{v,y} B(y)$ for unused v. From $\mathsf{Q}_y B(y)$, case introduction uses QE with its local base assumption to supply the successor premise. Conversely, DE opens $\mathsf{D}_{v,y} B(y)$ to the identical $\mathsf{Q}_y B(y)$ assumption, which is its own conclusion by hypothesis use. This argument precedes the reconstruction of ordinary logical connectives. An implementation that keeps Q only as an internal clause must say so; the main checker here uses the full formula-context interface.

2.3 Resources available before any arithmetic bootstrap

The following derivations use neither an ordinary selector nor pairing.

Lemma 2.1 (Pre-bootstrap equality, cases, and induction)**.** *The native calculus derives reflexivity and the ordinary equality laws, and it admits*

$$\frac{\Gamma \vdash F(0) \quad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)}, \qquad \frac{\Gamma \vdash E(0) \quad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}, \qquad \text{(NC, EI)}$$

where F is arbitrary, E is a literal equation, and z has the usual freshness condition.

Proof. For any term t, RS with constant update t gives $r_t = t$ for $r_t := R(1, 0; u.t)$. Transport that equation through the motive $v = t$, using a second copy as the minor, to obtain $t = t$. Reflexivity and transport give symmetry, transitivity, and congruence.

For NC, case-introduce $\mathsf{D}_{v,y} F(y)$ with unused witness variable v, ignoring its local base assumption in the successor premise. DE supplies $\mathsf{Q}_y F(y)$; the given base and QE produce $F(t)$. For EI, use progress introduction on $\mathsf{D}_{v,y} E(y)$ with unused v, then the same DE/QE construction and the given base. All local parameters can be renamed fresh before performing the construction. □

Throughout the new bootstrap, a numerical case split means NC or a finite split derived from it. It does not presuppose ordinary disjunction or induction on a disjunctive range assertion. Every induction invariant in that bootstrap is a literal equation with passive parameters fixed.

3 A scalar successor-seeded recursor is sufficient

This section gives an internal predecessor construction for the scalar successor-seeded recursor. Its update has only the previous state as a local variable. It does not receive the old counter. The construction first exchanges the numerical states zero and one to manufacture zero-start iteration. It then uses affine, rather than ordinary, addition in a factor-map construction. This avoids assuming either a predecessor or an unrestricted selector during the bootstrap.

3.1 Primitive syntax and available reasoning

The numerical terms are variables, 0, and

$$R(n, a; u.h).$$

Only u in the update h is bound. All its other variables are passive parameters. Every body is fixed finite earlier syntax. Write

$$Sa := R(0, a; u.0), \qquad 1 := S0,$$

and postulate precisely the two computation schemes

$$R(0, a; u.h) = Sa, \tag{R0}$$

$$R(Sn, a; u.h) = h[R(n, a; u.h)/u]. \tag{RS}$$

Thus successor is an abbreviation, and the zero equation for its chosen defining body is tautological. No numerical function variable or uniform interpreter is introduced.

The bootstrap uses equality transport and the following two derived rules, only for literal equations E:

$$\frac{\Gamma \vdash E(0) \qquad \Gamma \vdash E(Sz)}{\Gamma \vdash E(t)}, \qquad \frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}. \tag{NC, EI}$$

Here z is fresh for the retained context and fixed parameters. These rules follow directly from native DI-case and DI-progress, respectively, with an unused witness variable, followed by DE and QE. They therefore precede the reconstruction of ordinary logical connectives. Reflexivity is also available before any selector: (RS) gives $R(S0, 0; u.t) = t$ for a constant update, and transport of that equation along itself gives $t = t$. Symmetry, transitivity and term congruence then follow by transport.

Every induction below is an application of EI to one displayed equation, with its other parameters fixed. Finite-valued case analyses are abbreviations for repeated NC and substitution of already proved equations. They do not invoke induction on a disjunction, a quantified formula, or a simultaneous conjunction of equations.

3.2 A finite orbit that exchanges zero and one

Define the restricted selector and zero test

$$C^+(n, a, b) := R(n, a; u.b), \qquad Z(n) := C^+(n, 0, 0).$$

Computation gives

$$C^+(0,a,b) = Sa, \quad C^+(Sn,a,b) = b, \qquad Z(0) = 1, \quad Z(Sn) = 0.$$

The superscript emphasizes that this is not yet the ordinary selector. Its zero branch adds one to its supplied value.

Put

$$\epsilon(n) := R(n, 0; u.Z(u)), \qquad B(a,b) := Z\big(C^+(a, 0, Z(b))\big).$$

Thus $\epsilon(0) = 1$ and $\epsilon(Sn) = Z(\epsilon(n))$. Numerical cases on its counter and on the argument of Z show that $\epsilon(n)$ is always one of the two Boolean values. On Boolean arguments, B has the conjunction table. Consequently

$$\epsilon(SSn) = \epsilon(n).$$

No assertion about even or odd representations of arbitrary numbers is needed for this equation.

Define

$$q(u) := B\big(\epsilon(u), Z(Z(u))\big), \qquad A(n) := R(n, 1; u.q(u)), \qquad O(n) := Z\big(\epsilon(A(n))\big).$$

Finite computation gives $q(2) = 1$ and $q(1) = q(0) = 0$. Hence

$$A(0) = 2, \quad A(1) = 1, \quad A(SSn) = 0; \qquad O(0) = 0, \quad O(1) = 1, \quad O(SSn) = 0.$$

For $A(SSn) = 0$, the base is the two-step computation; its induction step applies q and uses $q(0) = 0$. The three equations for O then follow by substitution and finite computation.

Now set

$$j(u) := C^+\big(O(u), C^+(u, 0, u), 0\big), \qquad \tau(n) := R(n, 0; u.j(u)).$$

The preceding equations give

$$j(0) = 2, \qquad j(1) = 0, \qquad j(SSn) = SSSn.$$

It follows that

$$\tau(0) = 1, \qquad \tau(1) = 0, \qquad \tau(SSn) = SSn. \tag{1}$$

The third equation follows by EI: its base is $\tau(2) = j(\tau(1)) = j(0) = 2$; its step uses the displayed equation for $j(SSn)$. Two numerical cases on an arbitrary input now prove

$$\tau(\tau(n)) = n. \tag{2}$$

This proof requires neither predecessor nor successor injectivity.

3.3 Zero-start iteration by conjugation

For every fixed scalar update h, define

$$K(n; u.h) := \tau\big(R\left(n, 0; v.\tau\big(h[\tau(v)/u]\big)\right)\big). \tag{3}$$

Bound variables are renamed before this finite substitution. Its computation equations are

$$K(0; u.h) = 0, \qquad K(Sn; u.h) = h[K(n; u.h)/u]. \tag{4}$$

The base is $\tau(1) = 0$. At a successor, the two outer occurrences of τ cancel by (2). This gives a genuine zero-start scalar iterator for arbitrary fixed updates, not merely for the finite updates used to construct τ.

3.4 Affine arithmetic before predecessor

The shifted sum

$$T(x,y) := R(y,x;u.Su)$$

satisfies $T(x,0) = Sx$ and $T(x,Sy) = ST(x,y)$. Equation inductions give

$$T(0,y) = Sy, \quad T(Sx,y) = ST(x,y), \quad T(x,y) = T(y,x), \quad T(T(x,y),z) = T(x,T(y,z)). \tag{5}$$

For example, the first two identities are proved by induction on y; commutativity is then proved by induction on y, and associativity by induction on z. In particular, we have a commutative semigroup, not an addition operation with zero as identity.

Define

$$d(n) := K(n;u.SSu), \qquad D(n) := T(n,n), \qquad \Theta(n) := T(T(n,n),n).$$

Finite computation, (5), and EI give

$$\begin{aligned} d(0) &= 0, & d(Sn) &= SSd(n), & D(n) &= Sd(n), \\ D(0) &= 1, & D(Sn) &= SSD(n), & & \\ \Theta(0) &= 2, & \Theta(Sn) &= SSS\Theta(n), & & \\ \Theta(T(x,y)) &= T(\Theta(x),\Theta(y)). & & & & \end{aligned} \tag{6}$$

The last equality is simply reassociation and permutation of the six arguments after expanding the three copies of $T(x,y)$.

Let $N(n) := Z(\epsilon(n))$. The parity equations imply

$$\epsilon(D(n)) = 0, \quad \epsilon(\Theta(n)) = \epsilon(n), \quad N(\Theta(n)) = N(n), \quad N(Sn) = Z(N(n)). \tag{7}$$

For the first equation, induction follows the two successor increments of D. For the second, Θ advances by three successors, so both sides toggle once. The other equations follow by applying Z and its Boolean identities.

3.5 A ramp and an affine factor map

Define a three-state update and its zero-start orbit by

$$m(u) := C^+(u,0,C^+(\epsilon(u),1,0)), \qquad M(n) := K(n;u.m(u)).$$

Only the finite values

$$m(0) = 1, \qquad m(1) = 2, \qquad m(2) = 0$$

will be used. In particular, no general division algorithm or remainder theorem is assumed.

Set

$$F(n) := K(n;u.T(u,M(u))).$$

The central literal-equation induction is

$$M(F(n)) = N(n). \tag{8}$$

At zero, both sides are zero. If $N(n) = 0$, the next F state is $T(F(n),0) = SF(n)$, so its M value is $m(0) = 1 = N(Sn)$. If $N(n) = 1$, the next state is $T(F(n),1) = SSF(n)$, whose M value is

$m(m(1)) = m(2) = 0 = N(Sn)$. These are Boolean cases licensed by the preceding definitions and NC.

It follows that

$$F(Sn) = T(F(n), N(n)), \qquad F(SSn) = SSSF(n). \tag{9}$$

The second equation uses the two Boolean cases of $N(n)$ and $N(Sn) = Z(N(n))$. A further induction, using the two-step increments of D, gives

$$SF(D(n)) = \Theta(n). \tag{10}$$

Its base is $SF(1) = 2 = \Theta(0)$, and its successor step adds three successors to both sides.

The second scaling identity is

$$F(\Theta(n)) = T(\Theta(F(n)), N(n)). \tag{11}$$

Here is a detailed induction check. The zero case is $F(2) = 3 = T(2, 0)$. Write $X = \Theta(F(n))$ and $b = N(n)$. After using the induction equation, (9), and $N(\Theta(n)) = N(n)$, the new left side becomes

$$S^3T(T(X, b), b).$$

Using $F(Sn) = T(F(n), b)$ and the homomorphism law for Θ, the new right side becomes

$$T(T(X, \Theta(b)), Z(b)).$$

At $b = 0$ both terms reduce to S^5X; at $b = 1$ both reduce to S^7X, using $\Theta(0) = 2$ and $\Theta(1) = 5$. This is one equation induction with a Boolean case split, not induction with a varying parameter.

Now define

$$H(u) := C^+(\epsilon(u), F(u), u).$$

The essential factor equations are

$$H(0) = 0, \qquad H(D(u)) = \Theta(u), \qquad H(\Theta(u)) = \Theta(H(u)). \tag{12}$$

The first is a computation. The second follows from $\epsilon(D(u)) = 0$ and (10). For the third, $\epsilon(\Theta(u)) = \epsilon(u)$. If that Boolean value is 1, both sides are $\Theta(u)$. If it is 0, then $N(u) = 1$ and (11) gives

$$H(\Theta(u)) = SF(\Theta(u)) = ST(\Theta(F(u)), 1) = S^3\Theta(F(u)) = \Theta(SF(u)) = \Theta(H(u)).$$

The restricted selector is exactly sufficient: the transformed branch is a successor, and the retained branch is the unmodified state.

3.6 A strict comparator without general initialization

Define

$$E(x) := K(x; u.D(u)), \qquad V(x, y) := \epsilon\big(R(y, d(E(x)); u.H(u))\big).$$

We prove the three equations

$$V(x, 0) = 0, \qquad V(0, Sy) = 1, \qquad V(Sx, Sy) = V(x, y). \tag{13}$$

The standard-model meaning of V is the indicator of $x < y$, but that interpretation is not used in the proof.

At zero, the recursor state is $Sd(E(x)) = D(E(x))$, so the first equation follows from (7). For the second, $E(0) = d(0) = 0$ and the first update of the initial state $1 = D(0)$ is $H(1) = \Theta(0) = 2$. Moreover $H(2) = 2$. Equation induction on y therefore gives $R(Sy, 0; u.H(u)) = 2$, and $\epsilon(2) = 1$.

For the third equation, prove by EI on y, with x fixed,

$$R(Sy, d(E(Sx)); u.H(u)) = \Theta\big(R(y, d(E(x)); u.H(u))\big). \tag{14}$$

At $y = 0$, the left side is

$$H(Sd(E(Sx))) = H(D(E(Sx))) = \Theta(E(Sx)) = \Theta(D(E(x))),$$

which is the right side's initial value. The induction step applies H and then uses $H\Theta = \Theta H$. Applying ϵ to (14) proves the third equation of (13). This argument does not use an initializer-shifting rotation rule for R; such a rule would require precisely the missing predecessor.

3.7 Saturation constructs predecessor

Put

$$B_m(u) := C^+(Z(V(u, m)), u, u), \qquad W(t, m) := K(t; u.B_m(u)).$$

Because V is Boolean, $B_m(u)$ is Su when $V(u, m) = 1$ and is u when $V(u, m) = 0$. Direct numerical cases give

$$C^+(e, Sa, Sb) = SC^+(e, a, b).$$

Together with (13), this yields

$$B_{Sm}(Su) = SB_m(u), \qquad B_{Sm}(0) = 1.$$

Equation induction on t, with m fixed, now proves

$$W(St, Sm) = SW(t, m). \tag{15}$$

Its base is $B_{Sm}(0) = 1$; its step applies the first of the two preceding equations. Two separate equation inductions give

$$W(n, n) = n, \qquad V(n, n) = 0.$$

Consequently

$$W(Sn, n) = B_n(n) = n. \tag{16}$$

Define the finite scalar term

$$P(m) := K\big(m; u.C^+(Z(V(Su, m)), u, u)\big). \tag{17}$$

The zero equation is immediate. At parameter $m = Sn$, the update in (17) agrees pointwise with B_n, since $V(Su, Sn) = V(u, n)$. EI on a separate iteration counter t, keeping n fixed, therefore proves equality of the two zero-start trajectories. Only afterwards substitute $t = Sn$. Equation (16) then gives

$$P(0) = 0, \qquad P(Sn) = n. \tag{18}$$

This explicitly supplies the required body-extensionality argument; updates with different syntax were not silently identified.

3.8 Recovery of the ordinary scalar presentation

For every fixed initialized update define

$$I(n, a; u.h) := P\big(R\,(n, a; v.S\big(h[P(v)/u]\big))\big)\,. \tag{19}$$

Using (18) in the two computation cases proves

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The ordinary selector is now also available, for example as

$$C(n, a, b) := P(C^{+}(n, a, Sb)).$$

Its endpoints are exactly a and b. All ordinary logical gates and selectors can therefore be reconstructed after the arithmetic bootstrap. There is no circular appeal to full intuitionistic logic or an already available unrestricted selector in the predecessor proof.

Conversely, in the ordinary scalar presentation define

$$R(n, a; u.h) := I(n, Sa; u.h).$$

These descriptions satisfy (R0)–(RS). The two numerical translations preserve zero and successor, and each translated recursor agrees with the original by EI on its counter: their zero values agree and their step functions agree. With passive parameters fixed, this gives the termwise identities required for the two interpretation composites. The domain is unchanged. Combining this identity-domain equivalence with the established full scalar-iPRA bridge gives equiconsistency and bi-interpretability with iPRA.

Scope of the result. The reduction removes the independent successor constructor and the counter variable from the successor-seeded recursor's update language. It does not establish an absolute minimum for axiomatizations or a checker-size improvement by itself. A complete checker comparison must still count every syntax, substitution, scoping and proof rule.

4 From the new iterator to full intuitionistic PRA

The predecessor theorem supplies a direct compiler for ordinary initialized scalar iteration:

$$I(n, a; u.h) := P\big(R(n, a; v.S(h[Pv/u]))\big). \tag{20}$$

Writing $r(n) = R(n, a; v.S(h[Pv/u]))$, computation gives

$$I(0, a; h) = P(Sa) = a, \qquad I(Sn, a; h) = P(S(h[Pr(n)/u])) = h[I(n, a; h)/u].$$

The identity $P(Sx) = x$ applies at arbitrary source elements, so this is not a calculation restricted to standard counters. The final compiler is the conditional construction already stated in the shared-continuation draft[2]; the new bootstrap discharges its missing hypothesis.

In particular the unshifted selector $C(n, a, b) := I(n, a; u.b)$ satisfies $C(0, a, b) = a$ and $C(Sn, a, b) = b$. This is the point at which it becomes legitimate to reuse the existing Choice-DI logical decoding and scalar arithmetic completion. No unshifted selector is used to manufacture the predecessor that justifies it.

4.1 All ordinary intuitionistic logical operations

The inherited derivations are reproduced in Appendix A. Briefly, a formula gate $G_z A$ has a provable zero instance and a successor instance interderivable with A. On an equation,

$$G_z(s = t) := C(z, 0, s) = C(z, 0, t).$$

Gates commute with paired binders. A formula selector $M_z(A, B)$ has zero instance A and successor instance B; at equations it uses C on both sides. Different depths are aligned by the derived padding $A \dashv\vdash \mathsf{D}_{u,y} G_y A$, with unused u.

Let $\delta(z) = C(z, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$, so that $p(1) = 1$ and $p(2) = 0$. One full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z} M_z(A, B), \\ A \wedge_* B &:= \mathsf{D}_{u,z} G_z M_{p(z)}(A, B), \\ \exists_* x A(x) &:= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* x M_x(A, B), \\ \forall_* x A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &:= (0 = S0). \end{aligned}$$

All newly bound names are fresh. Case introduction and the gate/selector endpoints derive introduction and elimination for implication and existence. The instances $1, 2$ give the conjunction projections; NC gives disjunction elimination and the zero/successor split for universals. Under $0 = 1$, transport through C proves every equation. Structural induction on formulas, using the reconstructed introduction rules, gives explosion. This introduces no excluded-middle principle for quantified formulas.

4.2 Arithmetic completion and actual equality

After recovering I, define addition by $x + y = I(y, x; u.Su)$ and truncated subtraction by $d(x, y) = I(y, x; u.Pu)$. Applying P proves successor injectivity. The selector and the native absurdity bridge prove successor separation, and NC plus the logical macros gives numerical coverage.

The included equality-reflection proof constructs a numerical predicate for actual equality of arbitrary domain elements. Its essential residual invariant is a literal equation of the form

$$C(\mathrm{sg}(d(x, y)), x, y + d(x, y)) = x.$$

Its induction fixes x while varying y. From the resulting reflection theorem, every fixed quantifier-free formula has a characteristic equation; EI on that equation supplies its quantifier-free induction instance. Appendix B gives the full dependency-respecting argument. This step cannot be replaced by a finite test of an equality function on standard numerals.

Only after this completion is it legitimate to construct pairing by bounded square-root search. Appendix C includes terms π, L, H with $L(\pi(a, b)) = a$ and $H(\pi(a, b)) = b$. For a fixed ordinary recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

define

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))), \qquad f^* = H \circ V.$$

An equation induction proves $L(V(n,\vec{p})) = n$, and the projection identities prove both defining equations. Parameters stay fixed during that induction. The compiler recurses externally on each finite PR description and adds no uniformly total evaluator.

4.3 The two interpretations and both composites

For an ordinary first-order realization T_R, take intuitionistic equality logic with genuine falsity, R0, RS, literal-equation induction, and the boundary sentences

$$\forall x(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The boundary sentences belong to this ordinary realization. In the native presentation they are reconstructed from NC and the logical macros, with $0 = S0$ designated as absurdity. Positive computation equations alone would also admit a singleton structure; the falsity bridge is not omitted.

Interpret each fixed scalar R description in iPRA by primitive recursion with base Sa and update $h(u,\vec{p})$ that ignores the old counter. R0 and RS, the boundary properties, and EI are then iPRA theorems. Conversely, interpret every fixed PR description in T_R by the just-proved I compiler and pairing construction. Equality reflection and its characteristic equations validate all induction instances. Both interpretations leave the numerical domain unrestricted and retain actual equality.

For the stronger return condition, fix any two scalar recursions with the same seed and pointwise provably equal updates. Literal-equation induction comparing their values proves equality at every internal counter. The same uniqueness proof works for ordinary primitive recursion. External induction on the finite descriptions shows that each double translation has the same defining equations as the original operation. Internal uniqueness identifies them. This also justifies coherence of the staged descriptions under capture-free substitution.

The logical composites have the native equivalences

$$\mathsf{Q}_y B \dashv\vdash B(0) \Rightarrow_* \forall_* y B(y), \qquad \mathsf{D}_{x,y} B \dashv\vdash \exists_* x(B(x,0) \Rightarrow_* \forall_* y B(x,y)).$$

For the forward D direction use DE, assume the base, and apply QE at an arbitrary fresh parameter. For the reverse direction eliminate the existential and use case DI, obtaining its successor premise from the implication and universal eliminations. The Q argument is the same without the existential. Induction on finite native formulas gives the full formula round trips, not merely a translation of a selected fragment.

Thus $x = y$ is internally a total bijection preserving every numerical operation in both composites. Contradictions translate in both directions by the explicit falsity bridge. The effective finite proof translations give equiconsistency. No assertion that either theory proves its own consistency is involved. This proves Theorem 1.1.

5 Proof checking, source cost, and the precise evidence boundary

5.1 Generating the source of a computation

The smallest current comparator uses numerical equations as generated local assumptions. We inherit that principle, but change the arithmetic packet from a proposed source requiring a zero/successor shape check to an old recursive value and a choice of computation clause. The checker itself

constructs the new source counter. This avoids asking the same syntax both to provide a recursion argument and to certify that it has the required successor shape.

For canonical numerical data n, a, h, an arithmetic packet supplies the three-field description $s = [n, a, h]$ and a flag $b \in \{0, 1\}$. Its generated equation is

$$\begin{aligned} b = 0 : &\quad R(0, a; u.h) = Sa, \\ b = 1 : &\quad R(Sn, a; u.h) = h[R(n, a; u.h)/u]. \end{aligned}$$

The counter in the first line is generated as zero; the supplied n is unused. In the second line the supplied n is the predecessor, so it can be any well-scoped numerical term. In particular, it is not itself required to be a unary raw tree. This is a change in certificate format; old arithmetic packets must be translated.

The exact arithmetic branch is

```
n, t, v = s
h = [[u and [n], t, v], i(v, s, 1) if u else [t]]
```

Here the local `u` is the packet flag, `t` is the seed, and `v` is the fixed update body. These local Python names are not extra mathematical update variables. For a true flag, `[n]` is the successor abbreviation; for canonical false flag zero, the source counter is zero. No old-counter argument has been added to the update.

5.2 The entire trusted program

The following is a complete standalone program, including input, scope opening, rule checking, and the public verdict. It imports only the standard JSON parser. The numerical evaluator, proof producer, measurement script, and arithmetic bootstrap are not imported.

```
from json import load
k = 0
i = lambda a, t=None, d=0: [i(b, t, d + b' \x02\x01\x04'[len(a)]) for b in a] if
    isinstance(a, list) else a if k <= a < d else d % 2 and i(t, d=1)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    if m:
        c(m, g)
        if m in ([s, u], not s and [t]) and i(t, u) == a:
            a = i(t, s)
        h = m[2:]
    elif t:
        h = a = i(a[2], t, 2)
    else:
        n, t, v = s
        h = [[u and [n], t, v], i(v, s, 1) if u else [t]]
    k -= 1
    c(i(a, [k]), g + [i(h, k if m or len(a) % 2 == m else 0)])
try:
    q = load(open(0)).pop
    c(q(), [])
except:
    print('REJECT')
    raise
print('ACCEPT')
```

The public input is one JSON array. Its final element is the closed native goal; preceding four-field packets $[m, s, u, t]$ are popped from the end. A hypothesis leaf consumes no packet. An unread prefix is inert. Normal completion prints `ACCEPT` and exits zero. Failure prints `REJECT` and exits nonzero, retaining an exception traceback. Submitted strings are data and are never executed as code.

5.3 Exact raw syntax and opening

Canonical syntax uses arrays of arity at most three and integer leaves. The same raw tree is decoded at a numerical or formula sort:

Arity	Numerical meaning	Formula meaning	Depth increment
0	zero	$0 = 0$	no children
1	successor abbreviation	Q	2
2	zero, with checked annotations	equality	1
3	scalar R	D	4

Unary numerical notation expands to $R(0, a; u.0)$; it is not a further primitive operation in the mathematical theory. The three-field update has two raw aliases of its single state variable. They do not name independent state and counter arguments.

More explicitly, with nearest-first protected numerical environment ρ, the nontrivial numerical clauses are

$$\begin{aligned} T_\rho([p]) &= S(T_{0,\rho}(p)), \\ T_\rho([A, B]) &= 0, \\ T_\rho([n, a, h]) &= R(T_{0,0,\rho}(n), T_{0,0,\rho}(a); u.T_{u,u,\rho}(h)). \end{aligned}$$

Annotations in the second line are checked even though their values are discarded. Formula decoding is

$$F_\rho([B]) = \mathsf{Q}_y F_{y,\rho}(B), \quad F_\rho([s, t]) = (T_\rho(s) = T_\rho(t)), \quad F_\rho([P, U, B]) = \mathsf{D}_{x,y} F_{y,x,\rho}(B).$$

The first two D fields are checked inert padding. Scalar formula leaves denote truth, and scalar numerical zero denotes zero.

The opener $i(a, t, d)$ traverses these finite trees. At q protected slots the formula depth is $2q$ and the numerical depth $2q + 1$. Positive coordinates $2j - 1, 2j$ refer to the same jth slot; negative coordinates name rigid parameters. The global floor k starts at zero and decreases at each shared continuation. A leaf with $k \leq a < d$ is retained. An exposed numerical leaf requests the image t, which is itself checked at numerical root; an exposed formula leaf becomes truth. A missing used numerical image fails. These operations are finite capture-free substitution, not numerical program evaluation.

The initial hypothesis test validates the meaningful numerical content of the goal before a fresh name is allocated. Transport minors and introduced witnesses are likewise checked before their continuation. The guarded opener rejects objects and strings used as syntax. The inherited harmless Boolean and finite-fractional coordinate aliases have their usual integer-floor decoding; canonical examples use only integer coordinates. Unused packet fields remain annotations.

5.4 Logical branches and the common continuation

If m is true, its major premise must first be proved. When $m = [s, u]$, the tested endpoint equality implements transport; when $s = 0$ and $m = [t]$, it implements specialization of Q. In either case the target opening must equal the requested goal before the goal changes to the source opening. The slice $m[2:]$ is empty at these two arities, hence supplies only truth. An unmatched endpoint request leaves the goal unchanged and also adds only truth. Its major proof is still checked. A three-field major instead contributes its opened unary closure, exactly as in paired elimination.

If m is false and the original t is true, the packet introduces D using witness t. Its opened matrix is the new goal. After fresh allocation, the goal is its successor instance. The expression `k if m or len(a) % 2 == m else 0` selects the current equation instance for canonical progress marker $m = 0$, and the zero instance for compound matrices and for canonical case marker $m = []$. A literal equation has raw arity two; a formula containing an equation is not thereby eligible for progress introduction.

The remaining branch generates a computation assumption as above. Every non-hypothesis instruction then allocates one name, appends one local formula, and follows the same continuation. Contexts are extended by fresh lists, so a major subproof cannot mutate its caller's retained assumptions.

5.5 Why a zero instance suffices

Lemma 5.1 (Known-instance discharge)**.** *Suppose w is absent from Γ, A and $\Gamma \vdash H(0)$. Then*

$$\Gamma, H(w) \vdash A \quad \Longrightarrow \quad \Gamma \vdash A.$$

Proof. Rename inner eigenparameters if necessary, substitute zero for w in the premise derivation, and cut the given proof of $H(0)$ against its remaining hypothesis. The end sequent is unchanged. These are admissible finite proof transformations of the stated calculus. □

A generated raw equation may acquire the new parameter when it is opened into the context. It therefore need not hold for every value of that parameter. We claim only the zero instance required by Lemma 5.1. For example, dummy zero slots in a copied seed can open to w on one side. Substitution $w = 0$ restores their intended value.

Lemma 5.2 (Zero completion of raw slots)**.** *Opening extra exposed numerical slots to one fresh parameter and then substituting zero for that parameter agrees with interpreting those slots as dummy zeros. Existing rigid parameters and genuinely internal bound coordinates keep their values.*

Proof. Induct on the finite raw tree using the displayed two-sorted decoder. At a numerical coordinate the alternatives are an unchanged protected or rigid coordinate, or one of the specified zero slots. Formula scalar leaves are truth. Successor adds a dummy zero in both readings; numerical binary nodes denote zero independently of their checked annotations. For a recursion node, compare its two dummy zero slots in the counter and seed, and keep the two aliases of its symbolic state equal in the body. Apply the induction hypothesis to its three children. Formula constructors have the analogous capture-free argument. □

Lemma 5.3 (Generated-source computation is sound)**.** *Every computation equation surviving the common opening has a provable zero instance in the primitive scalar-R calculus.*

Proof. For the zero flag, the generated source has counter zero. The seed on the source side is read beneath two dummy zero slots, while its copy inside the successor on the right has one. At the zero instance these readings agree by Lemma 5.2. Thus R0 proves the equation. The update is fixed checked syntax, and its value is irrelevant at counter zero.

For a true flag, the source counter is constructed as $[n]$. It therefore has the required successor shape without inspecting the shape of n. Consider the evaluation of $i(v, s, 1)$. If it requests its substitution image anywhere, the opener first validates the entire supplied s at numerical root. In particular the original update is checked under its two protected aliases. A coordinate beyond those aliases cannot be mistaken for the state: the same coordinate in this complete validation would request a missing image and fail. The counter and seed of the recursive value are checked as well. The additional dummy zero arising from the newly constructed successor leaves this validated counter unchanged under the zero completion. The right side is exactly the capture-free substitution of $R(n, a; u.h)$ for the one state variable. The equation is therefore an RS instance.

If no substitution image is requested, the update has no meaningful occurrence of either exposed state alias. Its value is independent of the old state; erased annotations make no difference to that claim. The generated source is still checked by the common opening. Its zero instance is an RS equation with a constant-in-state update, whose right side is the same checked update value. It does not require checking an unused recursive value. This distinction is precisely why generating

the source counter permits removal of the old eager predecessor-iterator check. No invalid equation is admitted merely because a proposed counter happened to be non-unary. □

Freshness of the old goal and context now permits known-instance discharge. The harmless added truth and unused parameter on a neutral logical continuation are removed by the same substitution-and-cut argument, using the derivable theorem $0 = 0$.

Theorem 5.4 (Written implementation correspondence)**.** *With unbounded resources, a successful finite checking trace has a closed decoded consequence of the stated scalar-R theory. Conversely, every finite canonical derivation has a finite certificate accepted by each of the three compact profiles.*

Proof. For soundness, induct on the successful finite trace. Hypothesis use, transport, closure specialization, and the paired rules have the stated opening and freshness interpretations. The generated-computation and administrative cases follow from the preceding lemmas. Major premises are checked before they can be used.

For completeness, serialize a finite derivation in the program's depth-first order, allocating names at every common continuation and respecting hypothesis closure. Replace R0 and RS leaves by their canonical generated equation followed by hypothesis use. Such equations are constant in the newly allocated parameter. For an RS leaf, the packet supplies the predecessor recursion description and flag one; for R0 it supplies flag zero. Logical packets are unchanged. A zero witness can use the well-scoped truthy numerical representation `[0,0]`, which decodes to zero; the rule does not require the represented number itself to be nonzero. This is an effective syntactic transformation, not an inference from tests. □

This is a written metatheorem and implementation argument. It is not a Lean or Rocq verification of the Python runtime. Actual finite parser, stack, and memory limits can reject sufficiently large valid proofs. The included symbolic certificates test concrete instances and targeted failure boundaries; their finite success does not establish the theorem.

The delivered run passed 248 complete-program executions: 31 packets (16 positive and 15 negative), four standalone whole-program profiles, and ordinary and optimized Python. Six targeted mutant executions confirm detection of the constructed-successor requirement, the literal-equation restriction, and the allocated-parameter boundary. Separately, 542 exact-definition arithmetic checks passed for the finite swap, zero iteration, affine map, comparator, saturation, predecessor, and initialized-iteration compiler. These finite numerical experiments are in the standard natural numbers; the nonstandard-domain conclusions come from the written induction proofs.

5.6 Whole-program measurements and matched controls

The unchanged inherited measurer removes comments and genuine docstrings, applies `ast.unparse`, appends one newline, and counts significant Python tokens, AST nodes, and UTF-8 bytes of the complete module. The recorded runtime is CPython 3.12.14. The public input and verdict code are included. No helper interpreter or theorem library is omitted from a measured kernel.

Matched complete program	Tokens	AST	Bytes
Old supplied-I source protocol	281	318	685
Direct scalar-R adaptation	283	320	687
New generated-source scalar R	279	319	686
New scalar R, byte profile	279	321	681
New scalar R, AST profile	282	314	708
New generated-source I control	277	317	684
New generated-source I, byte import	277	319	679

Generating the source saves four tokens, one AST node, and one byte against the direct scalar-R adaptation. The byte profile uses the equivalent standard import `import json`; that change saves five bytes and adds two AST nodes. The AST profile uses the inherited strict exception-based list opener and the straightforward driver assignment `c((q := load(open(0)).pop)(), [])`. The list guard `a + []` is retained; no new string or object syntax aliases are introduced. These profiles have the same successful values and public verdicts, apart from irrelevant traceback details on rejection.

The scalar-R family envelope is 279/314/681, compared with the pinned corpus envelope 281/315/685. Different programs attain these minima; none of the new programs individually beats every old coordinate. The reductions are approximately 0.71%, 0.32%, and 0.58%. The source-protocol change also improves the old arithmetic basis, as the matched controls show. In each matched protocol, the successor seed costs two additional tokens, two AST nodes, and two bytes. Thus the smaller numerical primitive inventory is a mathematical result; the small implementation savings arise from the separate generated-source proof interface and ordinary source organization.

Run `python3 code/measure_checkers.py` from the package root to refresh the complete measurements in `evidence/checker_metrics.json`. Exact source hashes accompany every row. The full pinned corpus inventory and source classification are supplied separately. Absolute shortestness and a universal lower bound are not claimed.

6 What the construction does and does not minimize

The established numerical inventory now has one nullary constructor, one scalar recursion constructor, and two computation schemas. It needs neither primitive successor nor the counter coordinate of the earlier successor-recursion update. Compared with initialized scalar iteration, the primitive successor constructor is absorbed into the seed law. Compared with the counter-aware successor-seeded basis, one update variable is removed. The same Choice-DI logical rules are retained.

This is a simplification of the theory's primitive specification, obtained at the cost of a substantially more elaborate predecessor proof. The complexity of that metatheorem is not hidden inside a trusted oracle: the checker checks only the stated two computation schemas and the stated logical rules. The theorem permits ordinary arithmetic proofs to be compiled into that kernel; the numerical evaluator in the archive is untrusted validation code.

The supplied corpus contains many previous reductions. We therefore make no literature-wide priority claim, no assertion that the complete kernel is absolutely shortest, and no inference from a small collection of passing examples to universal formal correctness. The mathematical advance claimed relative to the inspected corpus is the proved solution of its explicitly open scalar left-inverse problem. Any complete-checker size improvement is the separate measured result stated earlier.

6.1 Why the initializer remains present

Deleting the independent initializer from the numerical grammar is a different proposal. For example, a fixed-one-seed counter-aware recursor $J(0;h) = 1$, $J(Sn;h) = h(n, J(n;h))$, with only unary initial operations, has a one-coordinate majorization obstruction: each term is bounded by a unary function of one designated input. Addition and the arbitrary selector do not have that property. The same argument survives arbitrary unary primitive-recursive primitives. An inherited version of this obstruction is already mentioned in the archived predecessor sources; it is not advertised here as a new discovery.

The present scalar R retains an independent initializer parameter, and its shifted sum is unbounded in either of two arguments. It therefore escapes that obstruction for a concrete reason. The obstruction concerns term compilation on the same domain; it does not establish impossibility of all relational or domain-changing interpretations allowed by the task.

7 Reproduction and contents

The archive includes the editable modular source, a standalone expanded LaTeX source, the compiled report, complete checker source, explicit certificate tests, arithmetic experiments, frozen comparison sources, source measurements, provenance, and a SHA-256 manifest. Run

```
python3 reproduce.py
python3 build.py
```

from its extracted directory. The first command checks the distributed manifest and reproduces the validation checks. The second builds the PDF with shell escape disabled. Exact normalized source measurements use the recorded CPython version; the unchanged corpus normalizer is included. The checker uses only Python and its standard JSON library. The Python arithmetic experiments do not replace the written internal proof, and no Lean or Rocq verification of the entire interpretation theorem is claimed.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0,0;u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x,y) \qquad \Gamma, \langle B(a,0); y.B(a,y)\rangle \vdash F}{\Gamma \vdash F} \quad \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \quad \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)} \quad \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context* Γ *and native formula* F*,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x, y)$ and its transformed premise is

$$\pi : \quad \Gamma, E(t, 0), E(t, z) \vdash E(t, Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t, Sz)$ under $\Gamma, E(t, 0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t, y)$ and all retained data. Under $\Gamma, E(t, 0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t, y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t, 0); y.E(t, y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t, 0)$ supplies its antecedent. Use at Sz gives $E(t, Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y} E(x, y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y} E(x, y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t, y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of* H *followed by Use. If an old derivation has* N *nodes and* r *atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D *are derivable without using the progress choice.*

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0 A, \qquad G_{Sv} A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y} G_y A \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y} A, \mathsf{D}_{x,y} B) &:= \mathsf{D}_{x,y} M_v(A, B), \\ M_v(E, \mathsf{D}_{x,y} B) &:= \mathsf{D}_{x,y} M_v(G_y E, B), \\ M_v(\mathsf{D}_{x,y} A, E) &:= \mathsf{D}_{x,y} M_v(A, G_y E), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z} M_z(A, B), \\ A \wedge_* B &:= \mathsf{D}_{u,z} G_z M_{p(z)}(A, B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$
$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x\big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \qquad \text{(Paired composite)}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the

established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [5]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix 3, the selector is $C(n,a,b) = I(n,a;u.b)$, addition is $x+y = I(y,x;u.Su)$, and truncated subtraction is $d(x,y) = I(y,x;u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

B.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{21}$$

$$C(0,a,b) = a, \qquad C(Sx,a,b) = b. \tag{22}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{23}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{24}$$

and induction for each *literal equation* $E(n,\vec{p})$:

$$\forall \vec{p}\,\left[(E(0,\vec{p}) \wedge \forall n(E(n,\vec{p}) \longrightarrow E(Sn,\vec{p}))) \longrightarrow \forall n\, E(n,\vec{p})\right]. \tag{25}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (25) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

B.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u,0,1), \qquad K(u) = C(u,1,Pu). \tag{26}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{27}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{28}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (21). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (27); symmetry and (24) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{29}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{30}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{31}$$

To prove the range assertion, split (23). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (22) also give, for all u, v,

$$C(u,v,v) = v. \tag{32}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{33}$$

If $u = 1 = S0$, (28) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

B.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{34}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{35}$$

Using these equations and (25), we now derive

$$0 + y = y, \tag{36}$$
$$(Sx) + y = S(x + y), \tag{37}$$
$$d(Sx, Sy) = d(x, y), \tag{38}$$
$$d(Sa, a) = 1. \tag{39}$$

Each induction below has a single equation as its induction formula.

For (36), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (37), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (37) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{40}$$

Indeed, both sides are $S(y + r)$.

For (38), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (39). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (38) and the induction hypothesis.

B.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C\big(\mathrm{sg}(d(x,y)),\ x,\ y + d(x,y)\big) = x. \tag{41}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (36) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (32).

For the step, assume (41) at y and abbreviate

$$r = d(x,y), \qquad r' = Pr = d(x,Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (41) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (40) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (41). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa,b) = 1 \quad \longleftrightarrow \quad a = b. \tag{42}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (41). If $d(Sa,b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (39) give $d(Sa,b) = d(Sa,a) = 1$.

Define the equality characteristic by the fixed description

$$e(a,b) = \mathrm{sg}\big(K(d(Sa,b))\big). \tag{43}$$

The sign, countdown, and residual reflection lemmas give

$$e(a,b) = 0 \quad \longleftrightarrow \quad a = b, \tag{44}$$

$$e(a,b) = 0 \ \vee\ e(a,b) = 1. \tag{45}$$

In detail, $e(a,b)=0$ is equivalent to $K(d(Sa,b))=0$ by (30), which is equivalent to $d(Sa,b)=1$ by (33), and then to $a=b$ by (42). The range assertion is (29) applied to $K(d(Sa,b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a,b)=0$ branch, (44) gives $a=b$. In the $e(a,b)=1$ branch, an assumption $a=b$ would give $e(a,b)=0$ and therefore $0=1$, contradicting separation. Hence

$$\forall a\,\forall b\,(a=b\vee(a=b\to\bot)) \tag{46}$$

is a theorem, not an extra hypothesis.

B.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t}=e(s,t), \qquad b_\bot=1, \tag{47}$$
$$b_{A\wedge D}=C(b_A,b_D,1), \tag{48}$$
$$b_{A\vee D}=C(b_A,0,b_D), \tag{49}$$
$$b_{A\to D}=C(b_A,b_D,0). \tag{50}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$(b_A=0\vee b_A=1)\quad\text{and}\quad(b_A=0\longleftrightarrow A). \tag{51}$$

Here are the full connective checks. In every case $b_A=1$ implies $\neg A$: if A held, the already established implication $A\to b_A=0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (44)–(45). For falsity the range assertion is $1=0\vee 1=1$, with its right disjunct proved by reflexivity. Also $1=0\to\bot$ follows from separation and symmetry, while $\bot\to 1=0$ follows by explosion. Thus $1=0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A=0\vee b_A=1$. In the zero branch, A holds and $b_{A\wedge D}=b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A\wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D}=1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A\wedge D$; conversely an assumption $A\wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D}=0$.

Disjunction. In the branch $b_A=0$, the formula A holds and $b_{A\vee D}=0$. The range is immediate, and both $b_{A\vee D}=0$ and $A\vee D$ hold, proving their equivalence. In the branch $b_A=1$, we have $\neg A$ and $b_{A\vee D}=b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A\vee D$. Conversely, from $A\vee D$, the A alternative contradicts $\neg A$ and yields $b_D=0$ by explosion, while the D alternative yields $b_D=0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (51) for every quantifier-free formula.

B.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (51), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

C Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [6]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

C.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a,b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a,b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

C.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u,\vec{p})), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n,\vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

C.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

References

[1] Vladimir Reshetnikov, *ProveIt: docs/drafts*, snapshot inspected 10 November 2026. Commit: `4a04139595eadc3e8818f1df2a94838ed4851d83`.
Pinned `docs/drafts` directory on GitHub.

[2] *iPRA with a Shared Elimination Continuation*, `ipra_shared_continuation_2026-11-10.zip`, Section 9, in [1]. The scalar successor-seeded candidate and conditional initialization compiler are inherited; its left-inverse problem is solved here.

[3] *Successor from Recursion*, `ipra_successor_recursion_2026-11-09.zip`, in [1]. Counter-aware two-variable predecessor and PR compiler.

[4] *Numerical Axioms as Local Assumptions*, `ipra_generated_assumptions_2026-11-10.zip`, in [1]. The computation-supply interface, baseline checkers, and frozen arithmetic/logical appendices are inherited with file hashes recorded.

[5] Inherited *From Literal-Equation Induction to Full Intuitionistic PRA*, reproduced as `reference/equality_reflection.tex` in [4]. Included with attribution and exact source provenance.

[6] Inherited scalar pairing and primitive-recursion development, reproduced as `reference/primitive_recursion.tex` in [4]. Included with attribution and exact source provenance.

[7] Saeed Salehi, *Decidable Formulas of Intuitionistic Primitive Recursive Arithmetic*, Reports on Mathematical Logic 36 (2002), 55–61.
`https://rml.tcs.uj.edu.pl/rml-36/36-SALEHI.pdf`.

[8] Daniel E. Severin, *Unary Primitive Recursive Functions*, arXiv:cs/0603063, version 2 (2006). Background on restricted iteration bases; not a source of the whole-domain scalar bootstrap proved here.
`https://arxiv.org/abs/cs/0603063`.